

Clase 267 — Hacking de firmware

Parte: **13 — Seguridad móvil, IoT e inalámbrica** · Fuente: *Practical IoT Hacking* (Chantzis et al.) y OWASP FSTM 🕒 Duración estimada: **120 min** · Nivel: **Avanzado**

🎯 Objetivo

Aprender a obtener, desempaquetar y analizar el firmware de un dispositivo embebido para descubrir credenciales, claves, binarios vulnerables y backdoors, y a emular el firmware para pruebas dinámicas. El alumno recorrerá la metodología OWASP FSTM: obtención, extracción del sistema de ficheros, análisis estático, emulación y verificación de la cadena de actualización.

⚠️ **Nota ética:** trabaja solo con firmware de dispositivos propios o de descargas oficiales del fabricante para tu equipo, o con imágenes de práctica (DVRF, IoTGoat). No redistribuyas firmware con copyright.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

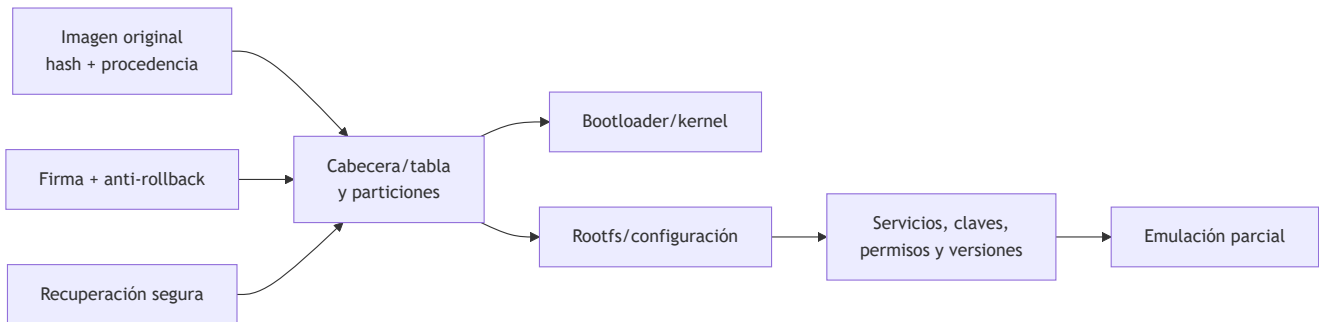
- 1. **Obtener** firmware por descarga oficial, volcado de flash o interceptación OTA.
- 2. **Extraer** el sistema de ficheros con `binwalk` y utilidades relacionadas.
- 3. **Analizar** estáticamente binarios, scripts y configuraciones en busca de secretos.
- 4. **Emular** el firmware o binarios con QEMU/FAT para pruebas dinámicas.
- 5. **Evaluar** la seguridad de la actualización (firma, cifrado) del firmware.
- 6. **Reportar** hallazgos con evidencia reproducible.

📖 Temas

#	Tema	Por qué importa
1	Obtención del firmware	Sin la imagen no hay análisis
2	Formatos y <code>binwalk</code>	Identifica y extrae componentes
3	Sistemas de ficheros (SquashFS, JFFS2)	Contienen binarios y configs
4	Búsqueda de secretos	Credenciales y claves embebidas
5	Análisis de binarios	Vulnerabilidades en servicios propios
6	Emulación con QEMU	Prueba dinámica sin el hardware
7	Seguridad de la actualización	Firma/cifrado del OTA

Firmware no es sinónimo de un único sistema de archivos

Una imagen puede contener cabecera del fabricante, bootloader, kernel, device tree, rootfs, configuración, particiones de recuperación y firma. `binwalk` reconoce patrones; un patrón es candidato, no prueba. La extracción conserva el original y su hash, identifica arquitectura y offsets y registra cada transformación.



El análisis busca cuentas, scripts de inicio, certificados, claves privadas, servicios y mecanismos de actualización. Una cadena encontrada puede ser una clave pública o dato de prueba; se valida uso. Contraseñas con hash requieren revisar algoritmo y política, no intentar acceso fuera del dispositivo propio.

Firmar actualizaciones protege autenticidad e integridad si la raíz de confianza y la verificación están protegidas. Anti-rollback evita instalar una versión antigua firmada pero vulnerable. NIST SP 800-193 organiza resiliencia en proteger, detectar y recuperar. Cifrar la imagen puede dificultar extracción, pero la clave y el código de descifrado deben existir en algún punto del producto.

La emulación rara vez reproduce hardware, drivers, secure elements o nube. Sirve para observar servicios bajo supuestos y debe marcarse como parcial. Una prueba que funciona en QEMU no demuestra el mismo impacto en el dispositivo.

Caso razonado: clave privada compartida

Dos imágenes oficiales contienen la misma clave privada de servidor. La evidencia se confirma comparando huellas en unidades propias. El problema no es solo «secreto en firmware»: una clave compartida impide identidad única y complica rotación. La corrección provisiona credenciales por dispositivo y define revocación y actualización.

Glosario operativo

Término	Definición útil
Rootfs	Sistema de archivos raíz usado por el runtime embebido.
Bootloader	Código inicial que prepara y carga etapas posteriores.
Anti-rollback	Control que rechaza versiones anteriores aun si están firmadas.
Root of trust	Componente protegido en el que comienza una decisión de confianza.
Emulación parcial	Ejecución aproximada que no reproduce todo el hardware.

✓ Criterio de dominio

El alumno domina la clase cuando conserva offsets y hashes, distingue detección de validación, explica firma y rollback, identifica una exposición con flujo de uso y limita las conclusiones de la emulación.

📖 Definiciones y características

- **binwalk:** herramienta que identifica y extrae firmas de archivos dentro de una imagen de firmware. Característica: automatiza la extracción del sistema de ficheros.
- **SquashFS:** sistema de ficheros comprimido de solo lectura muy común en IoT. Característica: se monta/extrae con `unsquashfs`.
- **Firmware Analysis Toolkit (FAT)/Firmadyne:** entorno de emulación de firmware con QEMU. Característica: permite ejecutar la interfaz web del dispositivo sin hardware.
- **Bootloader (U-Boot):** cargador de arranque de sistemas embebidos. Característica: su consola puede permitir volcar/flashear memoria.
- **Firma de firmware:** verificación criptográfica de integridad y origen del OTA. Característica: si falta, se puede instalar firmware modificado.
- **Entropía:** medida de aleatoriedad; alta entropía sugiere cifrado o compresión. Característica: ayuda a detectar particiones cifradas.

🧰 Herramientas y preparación

```
# Extracción
binwalk firmware.bin                # mapa de componentes
binwalk -e firmware.bin             # extraer recursivamente
unsquashfs squashfs-root.bin        # extraer SquashFS

# Búsqueda de secretos en el rootfs extraído
grep -rniE "password|passwd|admin|api[_-]?key|BEGIN .*PRIVATE KEY" squashfs-root/
find squashfs-root/ -name "*.conf" -o -name "shadow" -o -name "*.pem"

# Emulación
git clone https://github.com/attify/firmware-analysis-toolkit
./fat.py firmware.bin
```

- **Ghidra** para binarios extraídos; **qemu-user** para ejecutar binarios ARM/MIPS.

🧪 Laboratorio guiado

1. **Obtén el firmware:** descarga la imagen oficial de un router propio o usa DVRF/IoTGoat.
2. **Reconoce la imagen:** `binwalk firmware.bin` y observa entropía con `binwalk -E` para detectar cifrado.
3. **Extrae el rootfs:** `binwalk -e` o `unsquashfs`; localiza `/etc`, `/bin`, `/www`.
4. **Caza secretos:** busca `shadow` / `passwd`, claves privadas, certificados, cadenas de conexión y credenciales hardcodeadas.
5. **Analiza binarios:** identifica servicios (httpd, telnetd) y ábrelos en Ghidra buscando funciones peligrosas (`strcpy`, `system`).
6. **Emula:** ejecuta el firmware con FAT/Firmadyne y accede a su interfaz web para pruebas dinámicas.

7. **Revisa el OTA:** analiza si la imagen está firmada/cifrada e intenta comprender el mecanismo de actualización.
8. **Reporta:** documenta credenciales, claves y binarios vulnerables encontrados.

Ejercicios

1. Extrae el sistema de ficheros de una imagen de práctica y lista sus servicios de arranque.
2. Encuentra al menos una credencial o clave embebida y explica su impacto.
3. Calcula la entropía de la imagen y argumenta si está cifrada.
4. Identifica en Ghidra un uso peligroso de `system()` en un binario del firmware.
5. Emula el firmware y accede a su panel web.
6. Determina si la actualización del dispositivo está firmada.

Reto verificable

Analiza una imagen de firmware (propia o de práctica) y produce un informe con **al menos dos hallazgos:** una credencial/clave embebida y una función potencialmente vulnerable en un binario. **Criterio de aceptación:** para el binario, señalas la dirección/función concreta en Ghidra y explicas por qué el uso es peligroso; para la credencial, indicas el fichero exacto donde la hallaste.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>binwalk</code> no extrae nada	Firmware cifrado (alta entropía); busca la clave o vuelca el flash directamente
<code>unsquashfs</code> falla	Variante/compresión no estándar (LZMA/XZ); usa binwalk con soporte o sasquatch
Emulación no arranca la web	Falta NVRAM/hardware; usa Firmadyne que la simula
Binario no corre en QEMU	Arquitectura equivocada; usa <code>qemu-<arch></code> correcto y libs
Solo veo un blob	Partición única; segmentala por offsets de binwalk

Preguntas frecuentes

? ¿Cómo obtengo el firmware si no hay descarga oficial? Puedes volcarlo del chip de flash por SPI/JTAG (clase 268), interceptar una actualización OTA, o extraerlo desde la app/nube del fabricante.

? ¿Qué hago si el firmware está cifrado? Busca la clave en el bootloader o en una versión previa sin cifrar, o vuelca la RAM/flash tras el descifrado en tiempo de ejecución mediante acceso hardware.

? ¿La emulación reemplaza al hardware real? Para muchas pruebas de la interfaz web y servicios sí, pero periféricos y comportamiento dependiente de hardware requieren el dispositivo físico.

Referencias

- OWASP Firmware Security Testing Methodology: <https://github.com/scriptingxss/owasp-fstm>

- binwalk: <https://github.com/ReFirmLabs/binwalk>
- Firmware Analysis Toolkit / Firmadyne: <https://github.com/attify/firmware-analysis-toolkit>
- *Practical IoT Hacking*, caps. de firmware — Chantzis et al.



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Clase anterior

Clase 266 — Seguridad de IoT: panorama y superficie de ataque



Siguiente clase

Clase 268 — Análisis de hardware: UART, JTAG y SPI